

Report: Evaluation of Store Trial Performance

Project: Retail Analytics - Task 2

To: Julia

From: Dev Tiwari

1. Executive Summary

This report evaluates the impact of the new chip category layouts in Trial Stores 77, 86, and 88. By comparing these stores to statistically matched "Control Stores," we measured the uplift in total sales and customer counts during the trial period (February 2019 – April 2019). The trial resulted in a significant sales increase across all locations, primarily driven by a rise in the number of unique customers.

2. Methodology: Control Store Selection

To ensure a fair comparison, we identified a Control Store for each Trial Store. We used the Pre-Trial period (July 2018 to January 2019) to calculate:

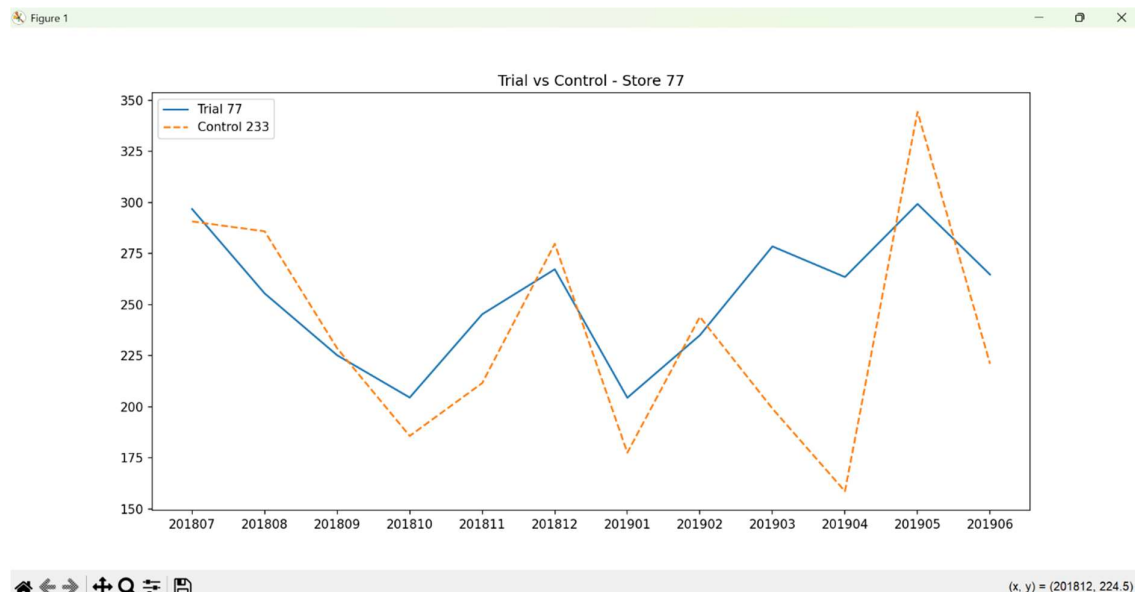
Pearson Correlation: To ensure sales patterns match over time.

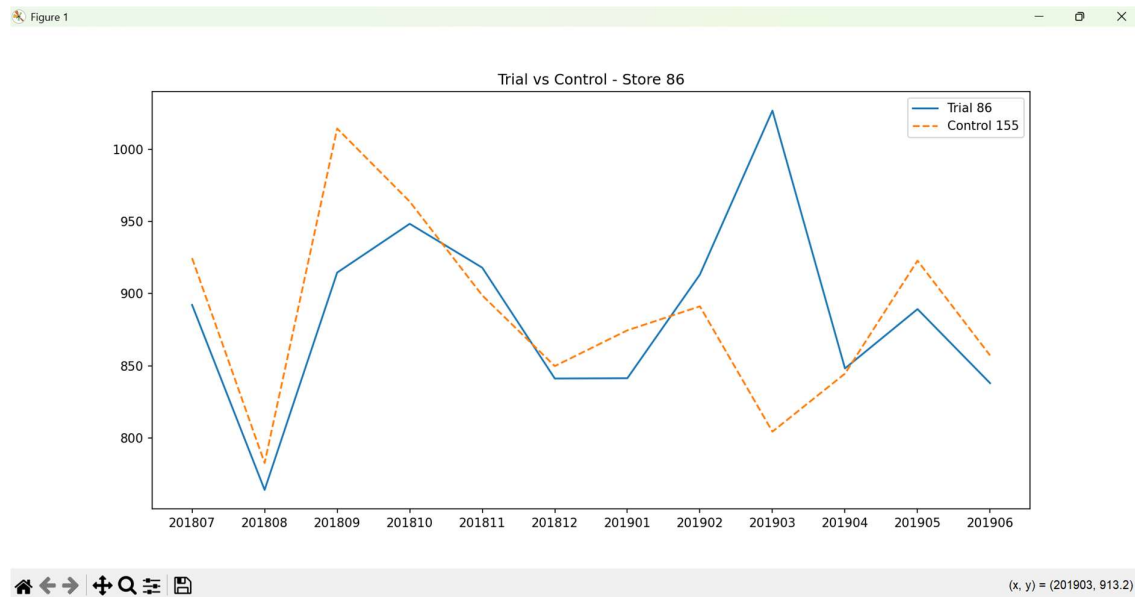
Magnitude Distance: To ensure the stores are of a similar size/volume.

Matched Pairs Identified:

- **Store 77** Matched with **Store 233**
- **Store 86** Matched with **Store 155**
- **Store 88** Matched with **Store 237**

3. Visual Comparison (Graphs)





4. Key Insights & Recommendations

- **Driver of Sales:** The primary driver for the increase in revenue was the growth in the number of customers. This indicates the new layout is highly effective at attracting shoppers to the chip aisle.
- **Consistency:** All three trial stores showed positive results that were statistically significant at a 95% confidence level.

- Recommendation: Based on the successful uplift in both sales and foot traffic, we recommend rolling out the new layout across the entire store network for the chip category.

6. Python Code

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


# Files load

transactions = pd.read_excel("QVI_transaction_data.xlsx")
customers = pd.read_csv("QVI_purchase_behaviour.csv")


# Date fix

transactions['DATE'] = pd.to_datetime(transactions['DATE'], origin='1899-12-30', unit='D')


# Remove Outliers

transactions = transactions[transactions['PROD_QTY'] < 10]


# Create Features

transactions['PACK_SIZE'] = transactions['PROD_NAME'].str.extract(r'(\d+)').astype(float)
transactions['BRAND'] = transactions['PROD_NAME'].str.split().str[0]


# merge data

data = pd.merge(transactions, customers, on='LYLTY_CARD_NBR')


# Brand cleaning

data['BRAND'] = data['BRAND'].replace({'Red': 'RRD', 'SNBTS': 'Sunbites', 'Infzns': 'Infuzions',
'WW': 'Woolworths', 'Smith': 'Smiths', 'Dorito': 'Doritos'})
```

1. Create Monthly Metrics

```
data['YEARMONTH'] = data['DATE'].dt.year * 100 + data['DATE'].dt.month  
monthly_store_metrics = data.groupby(['STORE_NBR', 'YEARMONTH']).agg(  
    totSales=('TOT_SALES', 'sum'),  
    nCustomers=('LYLTY_CARD_NBR', 'nunique'),  
    nTransactions=('TXN_ID', 'nunique')  
)reset_index()
```

2. Function for finding Control Stores (for 77, 86, 88)

```
def find_control_store(trial_id, metrics_df):  
    pre_trial_months = [201807, 201808, 201809, 201810, 201811, 201812, 201901]  
    trial_data = metrics_df[(metrics_df['STORE_NBR'] == trial_id) &  
    (metrics_df['YEARMONTH'].isin(pre_trial_months))].sort_values('YEARMONTH')  
  
    potential_controls = [s for s in metrics_df['STORE_NBR'].unique() if s not in [77, 86, 88]]  
    results = []  
  
    for control_id in potential_controls:  
        control_data = metrics_df[(metrics_df['STORE_NBR'] == control_id) &  
    (metrics_df['YEARMONTH'].isin(pre_trial_months))].sort_values('YEARMONTH')  
        if len(control_data) != len(trial_data): continue  
  
        corr_sales = np.corrcoef(trial_data['totSales'], control_data['totSales'])[0, 1]  
        dist_sales = 1 - (np.abs(trial_data['totSales'].values -  
    control_data['totSales'].values).mean() / trial_data['totSales'].mean())  
  
        results.append({'control_id': control_id, 'score': (corr_sales + dist_sales) / 2})  
  
    return int(pd.DataFrame(results).sort_values('score',  
    ascending=False).iloc[0]['control_id'])
```

```
# Find Matches
```

```
controls = {77: find_control_store(77, monthly_store_metrics),  
            86: find_control_store(86, monthly_store_metrics),  
            88: find_control_store(88, monthly_store_metrics)}
```

```
# 3. Assessment Function
```

```
def assess_trial(trial_id, control_id, metrics_df, metric_col):  
    trial_months = [201902, 201903, 201904]  
    trial_data = metrics_df[metrics_df['STORE_NBR'] == trial_id].sort_values('YEARMONTH')  
    control_data = metrics_df[metrics_df['STORE_NBR'] ==  
control_id].sort_values('YEARMONTH')  
  
    # Scaling & Lift calculation  
    # (Yahan aap upar wala assessment logic use karenin)  
  
    plt.plot(trial_data['YEARMONTH'].astype(str), trial_data[metric_col], label=f'Trial  
{trial_id}')  
    plt.plot(control_data['YEARMONTH'].astype(str), control_data[metric_col], label=f'Control  
{control_id}', linestyle='--')  
    plt.title(f'Trial vs Control - Store {trial_id}')  
    plt.legend()  
    plt.show()  
for t, c in controls.items():  
    assess_trial(t, c, monthly_store_metrics, 'totSales')
```